Walchand College of Engineering, Sangli

Department of Computer Science & Engineering

Computer Algorithm Laboratory

Week 4 Assignment — Divide and Conquer Strategy

Name : Atharv V Kulkarni

PRN : 245109074

Class : TY B.Tech CSE (B)

Question 1: Maximum Element in a Bitonic Array

Problem Statement:

Implement an algorithm to find the maximum element in an array which is first increasing and then decreasing, with time complexity O(log n).

Approach:

approach_1: scan every element linearly and track the maximum. tc: o(n) sc: o(1)

approach_2 (optimal): binary search comparing mid with its next element to decide which half holds the peak. tc: o(log n) sc: o(1)

Time Complexity: o(log n)

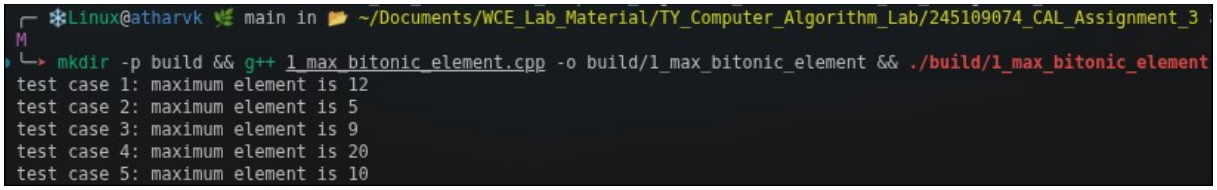
Space Complexity: o(1)

Code:

```
#include<iostream>
#include<vector>
using namespace std;
int maxBitonic(vector<int>&arr){
   int low=0,high=arr.size()-1;
   while(low<high){
      int mid=(low+high)/2;
      if(arr[mid]<arr[mid+1])
         low=mid+1;
      else
         high=mid;
   }
   return arr[low];
```

```
}
int main(){
    vector<vector<int>>tests={
        {1,3,8,12,4,2},
        {1,2,3,4,5},
        {9,7,5,3,1},
        {1,5,10,15,20,18,10,5},
        {2,4,6,8,10,9,7,3,1}
    };
    for(int i=0;i<(int)tests.size();i++)
        cout<<"test case "<<i+1<<": maximum element is "<<maxBitonic(tests[i])<<endl;
    return 0;
}
```

Output:



Question 2: Tiling Problem

Problem Statement:

Given an n×n board where n is of the form 2^k with k >= 1, and the board has one missing 1x1 cell, fill the remaining board using L-shaped tiles. An L-shaped tile is a 2x2 square with one 1x1 cell missing.

Approach:

approach_1 (optimal): divide the board into four quadrants, place one L-tile at the center covering the three quadrants that do not contain the missing cell, then recursively solve each quadrant treating the newly covered corner as its own missing cell.

Time Complexity: o(n squared)

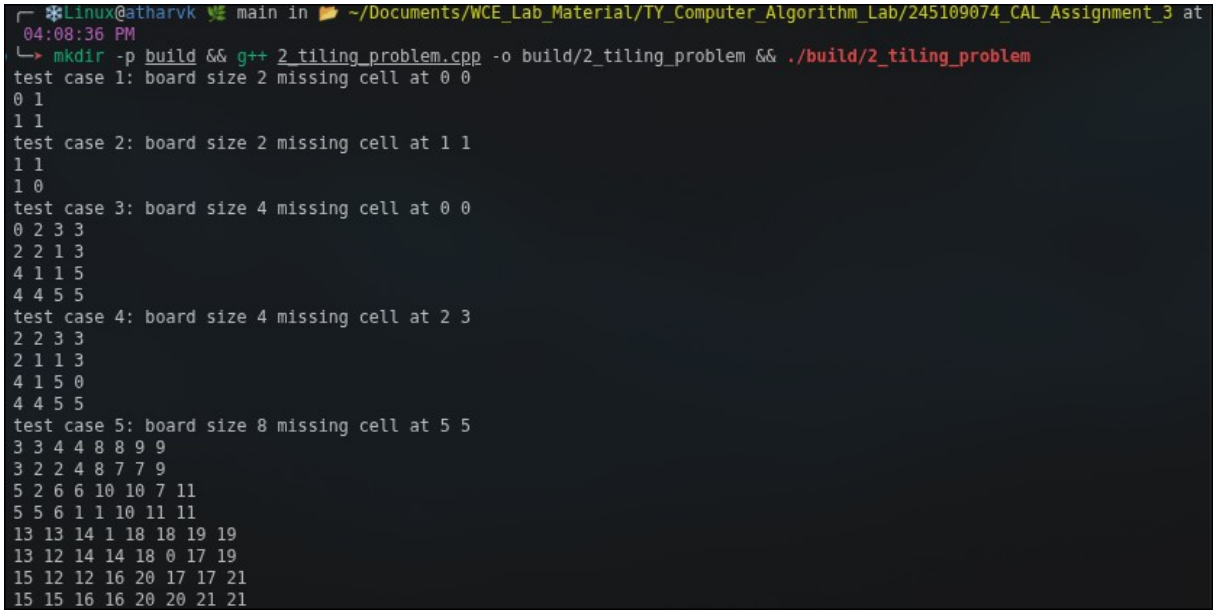
Space Complexity: o(n squared) for the board

Code:

```
#include<iostream>
#include<vector>
using namespace std;
void tileBoard(vector<vector<int>>&board,int size,int row,int col,int missingRow,int
missingCol,int&tileCount){
   if(size==1)
      return;
   int half=size/2;
   int cornertile=++tileCount;
   if(missingRow<row+half&&missingCol<col+half)
      tileBoard(board,half,row,col,missingRow,missingCol,tileCount);
   else{
      board[row+half-1][col+half-1]=cornertile;
      tileBoard(board,half,row,col,row+half-1,col+half-1,tileCount);
   }
   if(missingRow<row+half&&missingCol>=col+half)
      tileBoard(board,half,row,col+half,missingRow,missingCol,tileCount);
   else{
      board[row+half-1][col+half]=cornertile;
      tileBoard(board,half,row,col+half,row+half-1,col+half,tileCount);
   }
   if(missingRow>=row+half&&missingCol<col+half)
      tileBoard(board,half,row+half,col,missingRow,missingCol,tileCount);
   else{
      board[row+half][col+half-1]=cornertile;
      tileBoard(board,half,row+half,col,row+half,col+half-1,tileCount);
   }
   if(missingRow>=row+half&&missingCol>=col+half)
      tileBoard(board,half,row+half,col+half,missingRow,missingCol,tileCount);
   else{
      board[row+half][col+half]=cornertile;
      tileBoard(board,half,row+half,col+half,row+half,col+half,tileCount);
   }
```

```
}
void runCase(int n,int missingRow,int missingCol){
   vector<vector<int>>board(n,vector<int>(n,0));
   int tileCount=0;
   tileBoard(board,n,0,0,missingRow,missingCol,tileCount);
   for(int i=0;i<n;i++){
      for(int j=0;j<n;j++)
         cout<<board[i][j]<<" ";
      cout<<endl;
   }
}
int main(){
   int n[]={2,2,4,4,8};
   int mr[]={0,1,0,2,5};
   int mc[]={0,1,0,3,5};
   for(int i=0;i<5;i++){
      cout<<"test case "<<i+1<<": board size "<<n[i]<<" missing cell at "<<mr[i]<<" "<<mc[i]<<endl;
      runCase(n[i],mr[i],mc[i]);
   }
   return 0;
}
```

Output:



Question 3: The Skyline Problem

Problem Statement:

Given n rectangular buildings in a 2-dimensional city, each represented as a triplet (left, height, right), compute the skyline of these buildings eliminating hidden lines. A skyline is a collection of rectangular strips represented as pairs (left, height), with time complexity O(n log n).

Approach:

approach_1: brute force check every x coordinate against all buildings to find the tallest one covering it. tc: o(n squared) sc: o(n)

approach_2 (optimal): divide buildings into two halves, recursively compute each half's skyline, then merge the two skylines similar to the merge step of merge sort. tc: o(n log n) sc: o(n)

Time Complexity: o(n log n)

Space Complexity: o(n)

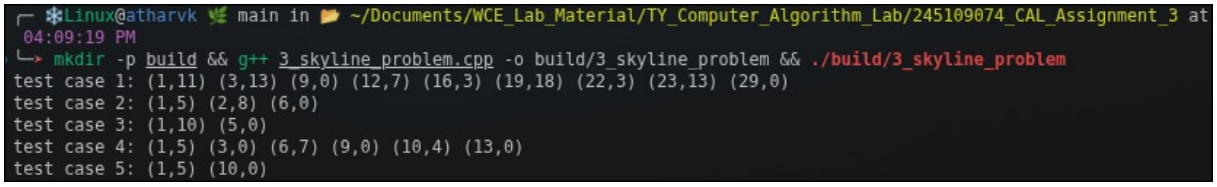
Code:

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
struct Building{
   int left;
   int height;
   int right;
};
bool cmpLeft(Building a,Building b){
   return a.left<b.left;
}
vector<pair<int,int>> merge(vector<pair<int,int>>&sky1,vector<pair<int,int>>&sky2){
   int h1=0,h2=0,i=0,j=0;
   vector<pair<int,int>>result;
   while(i<(int)sky1.size()&&j<(int)sky2.size()){
      int x;
      if(sky1[i].first<sky2[j].first){
         x=sky1[i].first;
         h1=sky1[i].second;
         i++;
      }
      else if(sky2[j].first<sky1[i].first){
         x=sky2[j].first;
         h2=sky2[j].second;
         j++;
      }
      else{
         x=sky1[i].first;
```

```
            h1=sky1[i].second;
            h2=sky2[j].second;
            i++;
            j++;
        }
        int maxh=(h1>h2)?h1:h2;
        if(result.empty()||result.back().second!=maxh)
            result.push_back({x,maxh});
    }
    while(i<(int)sky1.size()){
        result.push_back(sky1[i]);
        i++;
    }
    while(j<(int)sky2.size()){
        result.push_back(sky2[j]);
        j++;
    }
    return result;
}
vector<pair<int,int>> computeSkyline(vector<Building>&buildings,int low,int high){
    if(low==high){
        vector<pair<int,int>>s;
        s.push_back({buildings[low].left,buildings[low].height});
        s.push_back({buildings[low].right,0});
        return s;
    }
    int mid=(low+high)/2;
    vector<pair<int,int>>left=computeSkyline(buildings,low,mid);
    vector<pair<int,int>>right=computeSkyline(buildings,mid+1,high);
    return merge(left,right);
}
void runCase(vector<Building>buildings){
    sort(buildings.begin(),buildings.end(),cmpLeft);
    vector<pair<int,int>>skyline=computeSkyline(buildings,0,buildings.size()-1);
    for(auto p:skyline)
        cout<<"("<<p.first<<","<<p.second<<") ";
    cout<<endl;
}
int main(){
    vector<vector<Building>>tests={
        {{1,11,5},{2,6,7},{3,13,9},{12,7,16},{14,3,25},{19,18,22},{23,13,29},{24,4,28}},
        {{1,5,4},{2,8,6}},
        {{1,10,5}},
        {{1,5,3},{6,7,9},{10,4,13}},
        {{1,5,5},{5,5,10}}
    };
    for(int i=0;i<(int)tests.size();i++){
        cout<<"test case "<<i+1<<": ";
```

```
    runCase(tests[i]);
  }
  return 0;
}
```

Output:



Question 4: Closest Pair of Points

Problem Statement:

Implement an algorithm using divide and conquer to find the pair of points with the smallest Euclidean distance among n points given in a 2-dimensional plane, with time complexity O(n log n).

Approach:

approach_1: check every pair of points directly and keep the minimum distance. tc: o(n squared) sc: o(1)

approach_2 (optimal): sort points by x, recursively solve both halves, then check only the strip of points near the dividing line sorted by y for a closer pair. tc: o(n log n) sc: o(n)

Time Complexity: o(n log n)

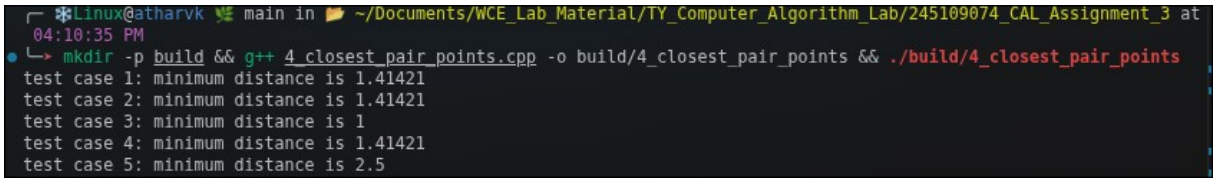
Space Complexity: o(n)

Code:

```
#include<iostream>
#include<vector>
#include<algorithm>
#include<cmath>
using namespace std;
struct Point{
   double x;
   double y;
};
bool cmpX(Point a,Point b){
   return a.x<b.x;
}
bool cmpY(Point a,Point b){
   return a.y<b.y;
}
double dist(Point a,Point b){
   return sqrt((a.x-b.x)*(a.x-b.x)+(a.y-b.y)*(a.y-b.y));
}
double bruteForce(vector<Point>&pts,int left,int right){
   double minDist=1e18;
   for(int i=left;i<=right;i++)
      for(int j=i+1;j<=right;j++){
         double d=dist(pts[i],pts[j]);
         if(d<minDist)
            minDist=d;
      }
   return minDist;
}
double stripClosest(vector<Point>strip,double d){
   double minDist=d;
   sort(strip.begin(),strip.end(),cmpY);
```

```
    for(int i=0;i<(int)strip.size();i++)
        for(int j=i+1;j<(int)strip.size()&&(strip[j].y-strip[i].y)<minDist;j++){
            double dd=dist(strip[i],strip[j]);
            if(dd<minDist)
                minDist=dd;
        }
    return minDist;
}
double closestUtil(vector<Point>&pts,int left,int right){
    if(right-left<=2)
        return bruteForce(pts,left,right);
    int mid=(left+right)/2;
    double midX=pts[mid].x;
    double dl=closestUtil(pts,left,mid);
    double dr=closestUtil(pts,mid+1,right);
    double d=(dl<dr)?dl:dr;
    vector<Point>strip;
    for(int i=left;i<=right;i++)
        if(fabs(pts[i].x-midX)<d)
            strip.push_back(pts[i]);
    double stripD=stripClosest(strip,d);
    return (stripD<d)?stripD:d;
}
double closestPair(vector<Point>pts){
    sort(pts.begin(),pts.end(),cmpX);
    return closestUtil(pts,0,pts.size()-1);
}
int main(){
    vector<vector<Point>>tests={
        {{0,0},{3,4},{1,1}},
        {{2,3},{12,30},{40,50},{5,1},{12,10},{3,4}},
        {{0,0},{1,0},{2,0},{3,0}},
        {{0,0},{5,5},{10,10},{1,1}},
        {{0,0},{0,3},{4,0},{4,3},{2,1.5}}
    };
    for(int i=0;i<(int)tests.size();i++)
        cout<<"test case "<<i+1<<": minimum distance is "<<closestPair(tests[i])<<endl;
    return 0;
}
```

Output:


Linux@atharvk main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_3 at
04:10:35 PM
mkdir -p build && g++ 4_closest_pair_points.cpp -o build/4_closest_pair_points && ./build/4_closest_pair_points
test case 1: minimum distance is 1.41421
test case 2: minimum distance is 1.41421
test case 3: minimum distance is 1
test case 4: minimum distance is 1.41421
test case 5: minimum distance is 2.5

Question 5: Strassen's Matrix Multiplication

Problem Statement:

Implement Strassen's algorithm using divide and conquer to multiply two n x n matrices A and B, where n is a power of 2, using fewer recursive sub-multiplications than the straightforward block-multiplication approach.

Approach:

approach_1: standard triple nested loop multiplication. tc: o(n cubed) sc: o(n squared)

approach_2 (optimal): split each matrix into four submatrices and combine them into 7 recursive multiplications instead of 8, following strassen's formulas. tc: o(n raised to log2 7), approximately o(n to the 2.81) sc: o(n squared)

Time Complexity: o(n raised to log2 7), approximately o(n to the 2.81)

Space Complexity: o(n squared)

Code:

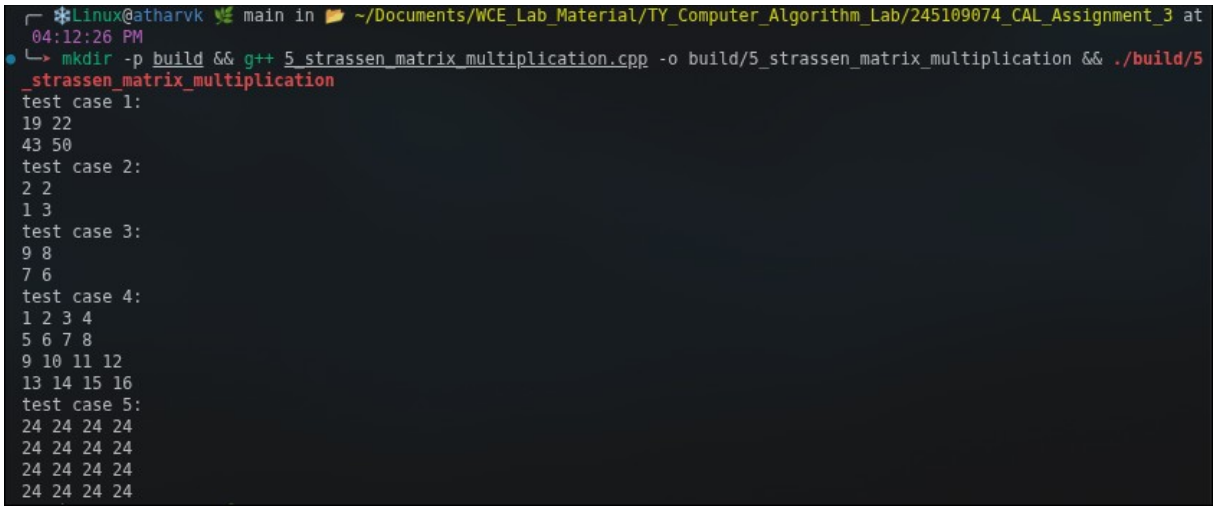
```
#include<iostream>
#include<vector>
using namespace std;
vector<vector<int>> addMatrix(vector<vector<int>>&A,vector<vector<int>>&B){
   int n=A.size();
   vector<vector<int>>C(n,vector<int>(n));
   for(int i=0;i<n;i++)
      for(int j=0;j<n;j++)
         C[i][j]=A[i][j]+B[i][j];
   return C;
}
vector<vector<int>> subMatrix(vector<vector<int>>&A,vector<vector<int>>&B){
   int n=A.size();
   vector<vector<int>>C(n,vector<int>(n));
   for(int i=0;i<n;i++)
      for(int j=0;j<n;j++)
         C[i][j]=A[i][j]-B[i][j];
   return C;
}
vector<vector<int>> strassen(vector<vector<int>>&A,vector<vector<int>>&B){
   int n=A.size();
   if(n==1){
      vector<vector<int>>C(1,vector<int>(1));
      C[0][0]=A[0][0]*B[0][0];
      return C;
   }
   int half=n/2;

vector<vector<int>>A11(half,vector<int>(half)),A12(half,vector<int>(half)),A21(half,vector<int>(half)
),A22(half,vector<int>(half));
```

```
vector<vector<int>>B11(half,vector<int>(half)),B12(half,vector<int>(half)),B21(half,vector<int>(half)
),B22(half,vector<int>(half));
   for(int i=0;i<half;i++)
      for(int j=0;j<half;j++){
         A11[i][j]=A[i][j];
         A12[i][j]=A[i][j+half];
         A21[i][j]=A[i+half][j];
         A22[i][j]=A[i+half][j+half];
         B11[i][j]=B[i][j];
         B12[i][j]=B[i][j+half];
         B21[i][j]=B[i+half][j];
         B22[i][j]=B[i+half][j+half];
      }
   vector<vector<int>>t1=addMatrix(A11,A22);
   vector<vector<int>>t2=addMatrix(B11,B22);
   vector<vector<int>>M1=strassen(t1,t2);
   vector<vector<int>>t3=addMatrix(A21,A22);
   vector<vector<int>>M2=strassen(t3,B11);
   vector<vector<int>>t4=subMatrix(B12,B22);
   vector<vector<int>>M3=strassen(A11,t4);
   vector<vector<int>>t5=subMatrix(B21,B11);
   vector<vector<int>>M4=strassen(A22,t5);
   vector<vector<int>>t6=addMatrix(A11,A12);
   vector<vector<int>>M5=strassen(t6,B22);
   vector<vector<int>>t7=subMatrix(A21,A11);
   vector<vector<int>>t8=addMatrix(B11,B12);
   vector<vector<int>>M6=strassen(t7,t8);
   vector<vector<int>>t9=subMatrix(A12,A22);
   vector<vector<int>>t10=addMatrix(B21,B22);
   vector<vector<int>>M7=strassen(t9,t10);
   vector<vector<int>>t11=addMatrix(M1,M4);
   vector<vector<int>>t12=subMatrix(t11,M5);
   vector<vector<int>>C11=addMatrix(t12,M7);
   vector<vector<int>>C12=addMatrix(M3,M5);
   vector<vector<int>>C21=addMatrix(M2,M4);
   vector<vector<int>>t13=addMatrix(M1,M3);
   vector<vector<int>>t14=subMatrix(t13,M2);
   vector<vector<int>>C22=addMatrix(t14,M6);
   vector<vector<int>>C(n,vector<int>(n));
   for(int i=0;i<half;i++)
      for(int j=0;j<half;j++){
         C[i][j]=C11[i][j];
         C[i][j+half]=C12[i][j];
         C[i+half][j]=C21[i][j];
         C[i+half][j+half]=C22[i][j];
      }
   return C;
```

```
}
void runCase(vector<vector<int>>A,vector<vector<int>>B){
    vector<vector<int>>C=strassen(A,B);
    for(int i=0;i<(int)C.size();i++){
        for(int j=0;j<(int)C.size();j++)
            cout<<C[i][j]<<" ";
        cout<<endl;
    }
}
int main(){
    vector<vector<int>>a1={{1,2},{3,4}};
    vector<vector<int>>b1={{5,6},{7,8}};
    vector<vector<int>>a2={{2,0},{1,2}};
    vector<vector<int>>b2={{1,1},{0,1}};
    vector<vector<int>>a3={{1,0},{0,1}};
    vector<vector<int>>b3={{9,8},{7,6}};
    vector<vector<int>>a4={{1,2,3,4},{5,6,7,8},{9,10,11,12},{13,14,15,16}};
    vector<vector<int>>b4={{1,0,0,0},{0,1,0,0},{0,0,1,0},{0,0,0,1}};
    vector<vector<int>>a5={{2,2,2,2},{2,2,2,2},{2,2,2,2},{2,2,2,2}};
    vector<vector<int>>b5={{3,3,3,3},{3,3,3,3},{3,3,3,3},{3,3,3,3}};
    cout<<"test case 1:"<<endl;
    runCase(a1,b1);
    cout<<"test case 2:"<<endl;
    runCase(a2,b2);
    cout<<"test case 3:"<<endl;
    runCase(a3,b3);
    cout<<"test case 4:"<<endl;
    runCase(a4,b4);
    cout<<"test case 5:"<<endl;
    runCase(a5,b5);
    return 0;
}
```

Output:


Linux@atharvk main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_3 at 04:12:26 PM
mkdir -p build && g++ 5_strassen_matrix_multiplication.cpp -o build/5_strassen_matrix_multiplication && ./build/5_strassen_matrix_multiplication
test case 1:
19 22
43 50
test case 2:
2 2
1 3
test case 3:
9 8
7 6
test case 4:
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
test case 5:
24 24 24 24
24 24 24 24
24 24 24 24
24 24 24 24